



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingenieria de Sistemas

Proyecto: Simulador de Bases de Datos

Curso: Calidad y Pruebas de Software

Docente: MAG. Patrick Cuadros Quiroga

Integrantes:

- Jhony Vargas Luque (2022075754)
- Abel Fernando Pacompia Ortiz (2023076797)

Tacna - Peru

2026

CONTROL DE VERSIONES

Version	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	APO, JVL	APO, JVL	P. Cuadros Q.	2026-04-25	Version inicial
2.0	APO, JVL	APO, JVL	P. Cuadros Q.	2026-06-21	Adaptacion a la implementacion final
2.1	APO, JVL	APO, JVL	P. Cuadros Q.	2026-07-04	Actualizacion con rutas, CI/CD y workflows actuales

Sistema Simulador de Bases de Datos

Documento de Arquitectura de Software

Version 2.1

INDICE GENERAL

I. [INTRODUCCION](#)

A. [Proposito](#)

B. [Alcance](#)

C. Definicion, siglas y abreviaturas

D. Referencias

II. Representacion Arquitectonica

- Diagrama de arquitectura general

A. Vista Escenarios

- Diagrama C4 de contexto

B. Vista Logica

C. Vista del Proceso

- Diagrama de Actividades del Sistema
- Diagrama de flujo tecnico

D. Vista del desarrollo

- Diagrama de Capas del Sistema Web
- Diagrama de Capas del Sistema Desktop
- Diagrama de paquetes o modulos

E. Vista Fisica

- Diagrama C4 de contenedores
- Diagrama de integracion con servicios externos

III. Objetivos y limitaciones arquitectonicas

A. Disponibilidad

B. Seguridad

- Diagrama de seguridad

IV. Analisis de Requerimientos

4.1 Priorizacion de requerimientos

A. Requerimientos funcionales

B. Requerimientos no funcionales

V. Vistas de Caso de Uso

Diagrama de Caso de Uso

VI. Vista Logica

A. Diagrama Contextual

VII. Vista de Procesos

A. Diagrama de Proceso Actual

B. Diagrama de Proceso Propuesto

VIII. Vista de Despliegue

A. Diagrama de Contenedor

IX. Vista de Implementacion

- A. Diagrama de Componentes - Sistema Web
 - B. Diagrama de Componentes - Sistema Desktop
 - X. Vista de Datos
 - A. Diagrama Entidad Relacion
 - XI. Calidad
 - A. Escenario de Seguridad
 - B. Escenario de Usabilidad
 - C. Escenario de Adaptabilidad
 - D. Escenario de Disponibilidad
 - E. Escenario de Escalabilidad
-

I. INTRODUCCION

A. Proposito (Diagrama 4+1)

Este documento describe la arquitectura del **Simulador de Bases de Datos**, una aplicacion web y desktop desarrollada con React, TypeScript, Vite, AlaSQL, IndexedDB, Firebase y Electron.

La arquitectura se presenta con un enfoque inspirado en el modelo **4+1 vistas**, considerando:

- Vista de casos de uso.
- Vista logica.
- Vista de implementacion.
- Vista de procesos.
- Vista de despliegue.

El objetivo es mostrar como se organizan los componentes, como interactuan entre si y que decisiones tecnicas sostienen los atributos de calidad del sistema.

B. Alcance

El documento cubre:

- Componentes frontend React.
- Estado global con Zustand.
- Motores simulados SQL, MongoDB y Redis.
- Persistencia local con IndexedDB y LocalStorage.
- Servicios Firebase para autenticacion, presencia, sesiones y roles.
- Simulador de carga.
- Panel administrativo.
- Empaquetado desktop con Electron.
- Workflows de GitHub Actions para rendimiento y despliegue.
- Diagramas PlantUML de arquitectura, procesos y despliegue.

No cubre una arquitectura de conexion a motores reales, porque la version actual ejecuta consultas de forma local y simulada.

C. Definicion, siglas y abreviaturas

Termino	Definicion
IDE	Entorno integrado para escribir y ejecutar consultas.
SQL	Lenguaje estructurado para bases de datos relacionales.
NoSQL	Modelos de datos no relacionales, como documentos o clave-valor.
AlaSQL	Motor SQL en memoria usado en el navegador.
IndexedDB	Base de datos local del navegador.
Zustand	Libreria de estado global para React.
Firebase Auth	Servicio de autenticacion.
RTDB	Firebase Realtime Database.
Electron	Plataforma para empaquetar aplicaciones web como desktop.
TPS	Transacciones o consultas por segundo, estimadas en el simulador.
CI/CD	Integracion y despliegue continuo mediante GitHub Actions.

D. Referencias

- FD01-EPIS. Informe de Factibilidad del Proyecto Simulador de Bases de Datos. Universidad Privada de Tacna, 2026.
- FD02-EPIS. Informe Vision del Proyecto Simulador de Bases de Datos. Universidad Privada de Tacna, 2026.
- FD03-EPIS. Informe SRS del Proyecto Simulador de Bases de Datos. Universidad Privada de Tacna, 2026.
- React Documentation: <https://react.dev/>
- TypeScript Documentation: <https://www.typescriptlang.org/docs/>
- Vite Documentation: <https://vitejs.dev/>
- Firebase Documentation: <https://firebase.google.com/docs>
- Electron Documentation: <https://www.electronjs.org/docs/latest/>

II. REPRESENTACION ARQUITECTONICA

Diagrama de arquitectura general

Diagrama Mermaid

```

flowchart TB
    Usuario((Usuario))
    Admin((Administrador))

    subgraph Cliente["Cliente"]
        Web["Aplicacion Web React"]
        Desktop["Electron Desktop"]
        IDB["IndexedDB"]
        LS["LocalStorage"]
    end

    subgraph Dominio["Dominio de aplicacion"]
        Store["Zustand Store"]
        Engine["Motor SQL/NoSQL simulado"]
        ImportExport["Importacion / Exportacion"]
        Simulator["Simulador de carga"]
    end

    subgraph Externo["Servicios externos"]
        Firebase["Firebase Auth / RTDB"]
        GitHub["GitHub Actions"]
        Pages["GitHub Pages / Hosting"]
    end

    Usuario --> Web
    Usuario --> Desktop
    Admin --> Web
    Web --> Store
    Desktop --> Store
    Store --> Engine
    Store --> Simulator
    Engine --> IDB
    Store --> LS
    ImportExport --> IDB
    Web --> Firebase
    Simulator --> Firebase
    GitHub --> Pages
  
```

A. Vista Escenarios

Diagrama C4 de contexto

Diagrama Mermaid

flowchart LR

Estudiante((Estudiante))

Docente((Docente))

Administrador((Administrador))

Sistema["Sistema: Simulador de Bases de Datos"]

Firebase["Sistema externo: Firebase"]

Hosting["Sistema externo: Hosting / GitHub Pages"]

Archivos["Archivos externos: SQL / CSV / JSON"]

Evidencias["Evidencias exportadas: CSV / JSON / Excel"]

Estudiante -->|Practica consultas| Sistema

Docente -->|Revisa evidencias| Sistema

Administrador -->|Monitorea sesiones y roles| Sistema

Sistema -->|Autenticacion / presencia| Firebase

Sistema -->|Publicacion landing/build| Hosting

Archivos -->|Importacion| Sistema

Sistema -->|Exportacion| Evidencias

Diagrama de Caso de Uso

Diagrama Mermaid

```
flowchart LR
    Usuario((Usuario))
    Administrador((Administrador))
    Docente((Docente))

    subgraph Sistema["Simulador de Bases de Datos"]
        UC1["Iniciar sesion"]
        UC2["Ejecutar consulta"]
        UC3["Importar datos"]
        UC4["Exportar evidencias"]
        UC5["Explorar esquema"]
        UC6["Simular carga"]
        UC7["Comparar motores"]
        UC8["Revisar historial"]
        UC9["Monitorear sesiones"]
        UC10["Gestionar roles"]
    end

    end

    Usuario --> UC1
    Usuario --> UC2
    Usuario --> UC3
    Usuario --> UC4
    Usuario --> UC5
    Usuario --> UC6
    Usuario --> UC7
    Usuario --> UC8
    Docente --> UC4
    Docente --> UC8
    Docente --> UC9
    Administrador --> UC1
    Administrador --> UC9
    Administrador --> UC10
```

B. Vista Logica

Diagrama de subsistemas

Diagrama Mermaid

```
flowchart LR
    subgraph Presentacion["Capa de Presentacion"]
        App
        SQLEditor
        ResultsPanel
        SchemaExplorer
        TopBar
        AdminApp
        SimulatorApp
    end

    subgraph Estado["Capa de Estado"]
        useStore
    end

    subgraph Dominio["Capa de Dominio / Motores"]
        sqlEngine
        queryLogger
        exportHelper
    end

    subgraph Persistencia["Capa de Persistencia"]
        idbStorage
        LocalStorage
    end

    subgraph Servicios["Capa de Servicios Externos"]
        auth
        presence
        simulatorSession
        adminUsers
        firebase
    end

    Presentacion --> Estado
    Presentacion --> Dominio
    Dominio --> Persistencia
    Presentacion --> Servicios
```


Diagrama de secuencia general

Diagrama Mermaid

```
sequenceDiagram
    actor Usuario
    participant UI as SQLEditor / TopBar
    participant Store as useStore
    participant Engine as sqlEngine
    participant Runtime as AlaSQL / Simuladores
    participant IDB as IndexedDB
    participant Logger as queryLogger
    participant Results as ResultsPanel

    Usuario->>UI: Solicita ejecutar consulta
    UI->>Store: Obtiene tab activo
    UI->>Engine: Ejecuta segun motor
    Engine->>Runtime: Procesa SQL/Mongo/Redis
    Runtime-->>Engine: Resultado
    Engine->>IDB: Persiste cambios si aplica
    Engine->>Logger: Registra ejecucion
    Engine-->>UI: QueryResult
    UI->>Store: Actualiza resultados
    Store->>Results: Renderiza estado
    Results-->>Usuario: Muestra resultados
```

Diagrama de colaboracion

Diagrama Mermaid

```
flowchart LR
    Usuario["Usuario"] -->|1. Ejecutar| TopBar["TopBar"]
    TopBar -->|2. Leer tab activo| Store["Store"]
    TopBar -->|3. Enviar query| SqlEngine["SqlEngine"]
    SqlEngine -->|4. Persistir cambios| IndexedDB["IndexedDB"]
    SqlEngine -->|5. Devolver resultado| Store
    Store -->|6. Actualizar UI| ResultsPanel["ResultsPanel"]
    ResultsPanel -->|7. Mostrar datos| Usuario
```

Diagrama de objetos

Diagrama Mermaid

```
flowchart TD
    Store["store: useStore"]
    Tab["tabSqlServer: EngineTab<br/>engine = sqlserver<br/>database = demo<br/>activeQueryPaneId = panel1"]
    Pane["pane1: QueryPane<br/>query = SELECT * FROM clientes"]
    Result["result: QueryResult<br/>rowCount = 25<br/>executionTime = 3ms"]
    Settings["settings: SimulationSettings<br/>networkLatency = 10<br/>connectionLimit = 50"]

    Store --> Tab
    Tab --> Pane
    Tab --> Result
    Store --> Settings
```

Diagrama de clases

Diagrama Mermaid

```
classDiagram
class EngineTab {
+string id
+EngineType engine
+string database
+string connection
+string query
+QueryResult results
}

class QueryPane {
+string id
+string query
+QueryResult results
+string[] messages
}

class QueryResult {
+string[] columns
+Record[] rows
+number rowCount
+number executionTime
+number memoryUsage
}

class SimulationSettings {
+number networkLatency
+number connectionLimit
+boolean simulateErrors
+number errorProbability
+IsolationLevel isolationLevel
}

class SchemaEntry {
+string dbName
+string[] tables
+number createdAt
}

class SimulatorSession {
+string id
+string name
+string engine
+string status
+number tps
}

class UseStore {
+EngineTab[] tabs
+SimulationSettings simulation
+DBEntry[] databases
+addTab()
+setTabResults()
+registerDatabase()
}
```

```

class SqlEngine {
    +executeSQL()
    +executeMongoQuery()
    +executeRedisCommand()
    +importTableFromSQL()
    +persistTables()
}

EngineTab "1" o-- "*" QueryPane
EngineTab --> QueryResult
UseStore --> EngineTab
UseStore --> SimulationSettings
SqlEngine --> QueryResult
SqlEngine --> SchemaEntry
SimulatorSession --> SimulationSettings

```

Diagrama de datos y persistencia

Diagrama Mermaid

```

flowchart LR
    subgraph IDB["IndexedDB: SimuladorBDS"]
        Tables["tables<br/>name, data"]
        Schemas["schemas<br/>dbName, DDL, tables"]
    end

    subgraph LS["LocalStorage"]
        Theme["theme"]
        Registry["simulador_bds_databases"]
        Session["session"]
    end

    subgraph RTDB["Firebase Realtime Database"]
        Users["users"]
        Presence["presence"]
        SimulatorSessions["simulator_web"]
    end

    IDB -->|metadatos complementarios| LS
    RTDB -->|sesion activa local| LS

```

C. Vista del Proceso

Diagrama de Actividades del Sistema

Diagrama Mermaid

flowchart TD

```

A["Usuario abre aplicacion"] --> B["React monta App"]
B --> C{"Sesion activa?"}
C -->|Si| D["Mostrar bienvenida / IDE"]
C -->|No| E["Mostrar LoginScreen"]
E --> F["Autenticar usuario"]
F --> D
D --> G["Inicializar base local"]
G --> H["Registrar presencia si Firebase existe"]
H --> I["Usuario selecciona motor"]
I --> J["Usuario ejecuta consulta"]
J --> K{"Motor SQL?"}
K -->|Si| L["executeSQL con AlaSQL"]
K -->|No| M{"MongoDB?"}
M -->|Si| N["executeMongoQuery"]
M -->|No| O["executeRedisCommand"]
L --> P["Actualizar resultados"]
N --> P
O --> P
P --> Q["Persistir cambios si aplica"]
Q --> R["Exportar o continuar practica"]

```

Diagrama de flujo tecnico

Diagrama Mermaid

flowchart TD

```

A["Evento UI: ejecutar/importar/exportar/simular"] --> B["useStore recibe accion"]
B --> C{"Tipo de accion"}
C -->|Consulta| D["sqlEngine procesa motor activo"]
C -->|Importacion| E["Import helper normaliza archivo"]
C -->|Exportacion| F["exportHelper genera salida"]
C -->|Simulacion| G["LoadSimulator calcula metricas"]
D --> H["Actualizar QueryResult"]
E --> I["Guardar tablas en IndexedDB"]
F --> J["Descargar archivo"]
G --> K["Publicar sesion si Firebase esta activo"]
H --> L["Renderizar ResultsPanel"]
I --> M["Actualizar SchemaExplorer"]
K --> N["Actualizar panel admin"]

```

D. Vista del desarrollo

Diagrama de Capas del Sistema Web

Diagrama Mermaid

```

flowchart TB
    subgraph SRC["src"]
        subgraph Components["components"]
            TopBar["TopBar.tsx"]
            SQLEditor["SQLEditor.tsx"]
            ResultsPanel["ResultsPanel.tsx"]
            SchemaExplorer["SchemaExplorer.tsx"]
            LoadSimulatorModal["LoadSimulatorModal.tsx"]
            AdminApp["AdminApp.tsx"]
        end

        subgraph Engines["engines"]
            sqlEngine["sqlEngine.ts"]
            exportHelper["exportHelper.ts"]
            queryLogger["queryLogger.ts"]
        end

        subgraph Store["store"]
            useStore["useStore.ts"]
        end

        subgraph DB["db"]
            idbStorage["idbStorage.ts"]
        end

        subgraph Lib["lib"]
            firebase["firebase.ts"]
            auth["auth.ts"]
            presence["presence.ts"]
            simulatorSession["simulatorSession.ts"]
            adminUsers["adminUsers.ts"]
        end

        subgraph Scripts["scripts"]
            perfTest["performance-test.mjs"]
            perfSummary["performance-summary.mjs"]
        end
    end

    subgraph Electron["electron"]
        main["main.cjs"]
        preload["preload.cjs"]
    end

    subgraph Workflows[".github/workflows"]
        performance["performance.yml"]
        pages["pages.yml"]
    end

    TopBar --> useStore
    SQLEditor --> sqlEngine
    sqlEngine --> idbStorage
    LoadSimulatorModal --> simulatorSession

```

```
AdminApp --> adminUsers
performance --> perfTest
performance --> perfSummary
pages --> landing["landing/"]
```

Diagrama de Capas del Sistema Desktop

Diagrama Mermaid

```
flowchart LR
    Usuario((Usuario)) --> UI["React UI<br/>App / Simulator / Admin"]
    Administrador((Administrador)) --> UI
    Electron["Electron Shell"] --> UI
    UI --> Store["Zustand Store"]
    UI --> Engine["Execution Engine<br/>AlaSQL + simuladores"]
    UI --> Export["Import / Export"]
    Engine --> IDB["IndexedDB"]
    Store --> LS["LocalStorage"]
    UI --> Auth["Firebase Auth"]
    UI --> RTDB["Firebase RTDB"]
```

Diagrama de paquetes o modulos

Diagrama Mermaid

```
flowchart LR
    subgraph UI["src/components"]
        LoginScreen
        TopBar
        EngineTabs
        SQLEditor
        ResultsPanel
        SchemaExplorer
        LoadSimulatorModal
        AdminApp
    end

    subgraph Store["src/store"]
        useStore
    end

    subgraph Engines["src/engines"]
        sqlEngine
        exportHelper
        queryLogger
    end

    subgraph DB["src/db"]
        idbStorage
    end

    subgraph Lib["src/lib"]
        firebase
        auth
        presence
        simulatorSession
        adminUsers
    end

    UI --> Store
    UI --> Engines
    Engines --> DB
    UI --> Lib
    Store --> DB
```

E. Vista Fisica

Diagrama C4 de contenedores

Diagrama Mermaid

```
flowchart TB
    Usuario((Usuario))
    Admin((Administrador))

    subgraph ContainerWeb["Container: React Web App"]
        UI["UI React"]
        Store["Zustand Store"]
        Engine["AlaSQL + simuladores MongoDB/Redis"]
    end

    subgraph ContainerDesktop["Container: Electron App"]
        Shell["Electron shell"]
        WebBundle["Build web embebido"]
    end

    subgraph ContainerStorage["Container: almacenamiento local"]
        IDB["IndexedDB"]
        LS["LocalStorage"]
    end

    subgraph ContainerFirebase["Container externo: Firebase"]
        Auth["Authentication"]
        RTDB["Realtime Database"]
    end

    Usuario --> UI
    Admin --> UI
    Shell --> WebBundle
    WebBundle --> UI
    UI --> Store --> Engine
    Engine --> IDB
    Store --> LS
    UI --> Auth
    UI --> RTDB
```


Diagrama de Contenedor

Diagrama Mermaid

```

flowchart TB
    subgraph Equipo["Equipo del Usuario"]
        subgraph Browser["Navegador"]
            ReactApp["React App"]
            IDB["IndexedDB"]
            LS["LocalStorage"]
        end
        subgraph Desktop["Electron Desktop"]
            Electron["Electron Shell"]
        end
    end

    subgraph Hosting["Servidor de Desarrollo / Hosting"]
        Vite["Vite Dev Server o Build Dist"]
        Pages["GitHub Pages<br/>landing/"]
        Netlify["Netlify<br/>dist/"]
    end

    subgraph Firebase["Firebase"]
        Auth["Authentication"]
        RTDB["Realtime Database"]
    end

    ReactApp -->|carga assets| Vite
    ReactApp -->|build produccion| Netlify
    Pages -->|publica| Landing["Landing estatica"]
    ReactApp -->|tablas / esquemas| IDB
    ReactApp -->|preferencias / sesion| LS
    ReactApp -->|login| Auth
    ReactApp -->|presencia / sesiones / roles| RTDB
    Electron -->|carga UI| ReactApp
  
```

Diagrama de integracion con servicios externos

Diagrama Mermaid

```

flowchart LR
    App["React / Electron App"]
    FirebaseAuth["Firebase Auth"]
    FirebaseRTDB["Firebase Realtime Database"]
    GitHubActions["GitHub Actions"]
    Hosting["GitHub Pages / Netlify"]
    Files["Archivos locales SQL/CSV/JSON"]
    Downloads["Descargas CSV/JSON/Excel/DDI"]

    Files -->|importar| App
    App -->|exportar| Downloads
    App -->|login y roles| FirebaseAuth
    App -->|presencia, sesiones, usuarios| FirebaseRTDB
    GitHubActions -->|build y pruebas de rendimiento| App
    GitHubActions -->|publica landing| Hosting
  
```

Escalabilidad:

- La ejecucion de consultas ocurre en el cliente, reduciendo carga de servidor.

- Firebase escala las funciones de presencia y sesiones.
- El build estatico puede desplegarse en Netlify, Firebase Hosting u otro hosting.
- La landing se despliega automaticamente en GitHub Pages desde `landing/`.
- GitHub Actions ejecuta build y pruebas de rendimiento antes de integrar cambios.

Disponibilidad:

- La aplicacion puede funcionar localmente para operaciones que no requieren Firebase.
- La persistencia IndexedDB permite conservar datos entre sesiones del mismo navegador.
- Las funciones admin dependen de disponibilidad de Firebase.

III. OBJETIVOS Y LIMITACIONES ARQUITECTONICAS

A. Disponibilidad

El IDE puede operar parcialmente sin Firebase para consultas locales, importacion, exportacion y persistencia en IndexedDB. Las funciones de login, presencia y administracion dependen de la disponibilidad de Firebase.

B. Seguridad

La arquitectura separa las funciones administrativas mediante autenticacion y roles. Las credenciales y variables de entorno de Firebase deben mantenerse fuera del repositorio publico.

Diagrama de seguridad

Diagrama Mermaid

```

flowchart TD
    Usuario((Usuario))
    Admin((Administrador))
    Login[LoginScreen]
    Auth[Firebase Auth]
    Roles[Validacion de rol]
    IDE[IDE principal]
    AdminPanel[Panel administrativo]
    RTDB["Firebase RTDB"]
    Local["IndexedDB / LocalStorage"]

    Usuario --> Login
    Admin --> Login
    Login --> Auth
    Auth --> Roles
    Roles -->|rol usuario| IDE
    Roles -->|rol admin| AdminPanel
    IDE --> Local
    AdminPanel --> RTDB
    AdminPanel -->|gestiona roles| RTDB
  
```

Restricciones Tecnicas

- La ejecucion SQL depende de AlaSQL.
- IndexedDB depende del navegador.
- Firebase requiere variables de entorno configuradas.
- Electron depende del sistema operativo objetivo.
- MongoDB y Redis son simulaciones, no servidores reales.

Restricciones Operacionales

- El usuario debe ejecutar `npm install` y `npm run dev` para desarrollo.
- Los datos locales pueden perderse si el navegador limpia IndexedDB.
- Las funciones admin requieren conexion y configuracion Firebase.
- El rendimiento de consultas depende del equipo cliente.

Restricciones del Negocio

- El sistema es academico y no reemplaza motores reales.
 - Las metricas de carga son didacticas.
 - No debe presentarse como benchmark real de bases de datos.
-

IV. ANALISIS DE REQUERIMIENTOS

4.1 Priorizacion de requerimientos

A. Requerimientos funcionales

ID	Requerimiento Arquitectonico
RF-A01	Ejecutar consultas SQL en memoria mediante un motor local.
RF-A02	Simular operaciones MongoDB y Redis.
RF-A03	Persistir tablas y esquemas en IndexedDB.
RF-A04	Gestionar estado de tabs, consultas, resultados y simulacion.
RF-A05	Importar archivos SQL, CSV y JSON.
RF-A06	Exportar resultados y esquemas en multiples formatos.
RF-A07	Simular carga con metricas de TPS, latencia, CPU, conexiones y errores.
RF-A08	Registrar presencia y sesiones cuando Firebase esta configurado.
RF-A09	Permitir panel administrativo con roles.
RF-A10	Construir version web y desktop.
RF-A11	Automatizar pruebas de rendimiento y despliegue de landing.

B. Requerimientos no funcionales

ID	Atributo	Decisiones Arquitectonicas
RNF-A01	Usabilidad	Layout tipo IDE, Monaco Editor, modales y acciones visibles.
RNF-A02	Rendimiento	Ejecucion local en memoria e IndexedDB para persistencia.
RNF-A03	Mantenibilidad	Separacion por componentes, motores, store, librerias y servicios.
RNF-A04	Portabilidad	Vite para web y Electron para desktop.
RNF-A05	Auditabilidad	Historial, logs, exportaciones y sesiones de simulador.
RNF-A06	Seguridad	Firebase Auth y roles para administracion.
RNF-A07	Extensibilidad	Configuracion central de motores y exportadores por modulo.
RNF-A08	Integracion continua	GitHub Actions valida build, rendimiento y despliegue.

V. VISTAS DE CASO DE USO

El diagrama de caso de uso resume las interacciones principales de Usuario, Docente y Administrador con el simulador.

Diagrama Mermaid

```

flowchart LR
    Usuario((Usuario / Estudiante))
    Docente((Docente))
    Admin((Administrador))

    subgraph Sistema["Simulador de Bases de Datos"]
        UC1(["Iniciar sesion"])
        UC2(["Seleccionar motor"])
        UC3(["Escribir consulta"])
        UC4(["Ejecutar SQL / NoSQL"])
        UC5(["Importar datos"])
        UC6(["Explorar esquema"])
        UC7(["Exportar resultados"])
        UC8(["Ver historial y logs"])
        UC9(["Simular carga"])
        UC10(["Comparar motores"])
        UC11(["Monitorear sesiones"])
        UC12(["Gestionar usuarios y roles"])
        UC13(["Revisar evidencias"])
    end

    end

    Usuario --> UC1
    Usuario --> UC2
    Usuario --> UC3
    Usuario --> UC4
    Usuario --> UC5
    Usuario --> UC6
    Usuario --> UC7
    Usuario --> UC8
    Usuario --> UC9
    Usuario --> UC10

    Docente --> UC9
    Docente --> UC10
    Docente --> UC13

    Admin --> UC11
    Admin --> UC12

    UC4 -. incluye .-> UC6
    UC5 -. actualiza .-> UC6
    UC9 -. genera .-> UC13
    UC11 -. requiere .-> UC1
    UC12 -. requiere .-> UC1

```

VI. VISTA LOGICA

A. Diagrama Contextual

El diagrama contextual muestra los subsistemas logicos internos y sus dependencias principales.

Diagrama Mermaid

```

flowchart TB
    Usuario((Usuario))
    Admin((Administrador))

    subgraph Sistema["Sistema Simulador de Bases de Datos"]
        UI["Interfaz React<br/>App / Simulador / Admin"]
        Store["Estado global<br/>Zustand"]
        Editor["Editor Monaco"]
        Results["Panel de resultados"]
        Schema["Explorador de esquema"]
        ImportExport["Importacion / Exportacion"]
        Load["Simulador de carga"]

        subgraph Core["Nucleo logico"]
            SQL["Motor SQL<br/>AlaSQL + preprocesadores"]
            Mongo["Simulador MongoDB"]
            Redis["Simulador Redis"]
            Metrics["Calculo de metricas"]
        end

        subgraph Data["Capa de datos"]
            IDBStorage["Persistencia IndexedDB"]
            LocalPrefs["Preferencias LocalStorage"]
        end

        subgraph Services["Servicios externos opcionales"]
            Auth["Firebase Auth"]
            RTDB["Firebase RTDB<br/>presencia / sesiones / roles"]
        end
    end

    Usuario --> UI
    Admin --> UI
    UI --> Store
    UI --> Editor
    UI --> Results
    UI --> Schema
    UI --> ImportExport
    UI --> Load
    Store --> Core
    Editor --> SQL
    Editor --> Mongo
    Editor --> Redis
    SQL --> IDBStorage
    Mongo --> IDBStorage
    Redis --> IDBStorage
    ImportExport --> IDBStorage
    Results --> ImportExport
    Schema --> IDBStorage
    Load --> Metrics
    Load --> RTDB
    UI --> Auth
    UI --> RTDB
    Store --> LocalPrefs

```

VII. VISTA DE PROCESOS

A. Diagrama de Proceso Actual

El proceso actual corresponde a la practica tradicional con instalacion de motores reales, configuracion manual y uso de herramientas separadas por tecnologia.

Diagrama Mermaid

flowchart TD

```
A["Inicio de practica de bases de datos"] --> B["Elegir motor requerido"]
B --> C["Buscar instalador / servicio"]
C --> D["Instalar motor local"]
D --> E["Configurar puertos, usuarios y credenciales"]
E --> F{"Configuracion correcta?"}
F -->|No| G["Corregir errores de instalacion"]
G --> E
F -->|Si| H["Instalar cliente o herramienta externa"]
H --> I["Crear base de datos y tablas"]
I --> J["Cargar datos de prueba"]
J --> K["Ejecutar consultas"]
K --> L{"Se requiere otro motor?"}
L -->|Si| B
L -->|No| M["Exportar evidencias manualmente"]
M --> N["Fin de practica"]
```

B. Diagrama de Proceso Propuesto

El proceso propuesto permite que el usuario acceda al simulador, seleccione motor, ejecute consultas, revise resultados y exporte evidencias desde un solo entorno.

Diagrama Mermaid

flowchart TD

```

A["Inicio de practica"] --> B["Abrir aplicacion web o desktop"]
B --> C["Iniciar sesion si corresponde"]
C --> D["Seleccionar motor simulado"]
D --> E["Tipo de entrada"]
E -->|Consulta manual| F["Escribir consulta en Monaco Editor"]
E -->|Archivo| G["Importar SQL, CSV o JSON"]
G --> H["Guardar tablas y esquemas en IndexedDB"]
F --> I["Ejecutar consulta"]
H --> I
I --> J["Motor seleccionado"]
J -->|SQL| K["Procesar con AlaSQL"]
J -->|MongoDB| L["Ejecutar simulador MongoDB"]
J -->|Redis| M["Ejecutar simulador Redis"]
K --> N["Mostrar resultados"]
L --> N
M --> N
N --> O["Actualizar esquema, historial y logs"]
O --> P["Requiere simulacion de carga?"]
P -->|Si| Q["Configurar usuarios, duracion y motor"]
Q --> R["Calcular TPS, latencia, CPU y errores"]
R --> S["Generar reporte de simulacion"]
P -->|No| T["Exportar resultados o base"]
S --> T
T --> U["Fin de practica"]

```

VIII. VISTA DE DESPLIEGUE

A. Diagrama de Contenedor

El diagrama de contenedor presenta la distribucion fisica y logica del sistema en navegador, escritorio, almacenamiento local, servicios externos y automatizacion.

Diagrama Mermaid

```

flowchart TB
    Usuario((Usuario))
    Admin((Administrador))
    Dev((Desarrollador))

    subgraph Client["Dispositivo del usuario"]
        Browser["Navegador moderno"]
        Electron["Aplicacion Electron"]
        IDB["IndexedDB<br/>tablas y esquemas"]
        LS["LocalStorage<br/>preferencias e historial"]
    end

    subgraph StaticHosting["Hosting estatico"]
        Landing["Landing page"]
        WebApp["Build Vite<br/>app.html / simulator.html / admin.html"]
    end

    subgraph AppContainer["Contenedor de aplicacion"]
        React["React UI"]
        Store["Zustand Store"]
        Engine["AlaSQL + simuladores<br/>MongoDB / Redis"]
        Exporter["Exportadores<br/>CSV / JSON / Excel / SQL"]
    end

    subgraph FirebaseCloud["Firebase"]
        Auth["Firebase Auth"]
        RTDB["Realtime Database<br/>presencia / sesiones / roles"]
    end

    subgraph CI["GitHub Actions"]
        Build["Build y validacion"]
        Performance["Pruebas de rendimiento"]
        Pages["Deploy landing"]
    end

    Usuario --> Browser
    Usuario --> Electron
    Admin --> Browser
    Browser --> WebApp
    Electron --> WebApp
    WebApp --> React
    React --> Store
    Store --> Engine
    Engine --> IDB
    React --> LS
    React --> Exporter
    React --> Auth
    React --> RTDB
    Dev --> CI
    CI --> Build
    CI --> Performance
    CI --> Pages
    Pages --> Landing
    Build --> WebApp

```

IX. VISTA DE IMPLEMENTACION

A. Diagrama de Componentes - Sistema Web

El sistema web se organiza en componentes React, store Zustand, motores simulados, persistencia local, servicios Firebase, scripts de rendimiento y workflows de GitHub Actions.

Diagrama Mermaid

```

flowchart TB
    Usuario((Usuario))
    Admin((Administrador))

    subgraph Web["Sistema Web React"]
        App["App.tsx"]
        AdminApp["AdminApp.tsx"]
        SimulatorView["Vista simulador<br/>simulator.html"]

        subgraph UI["Componentes de interfaz"]
            Login["LoginScreen"]
            TopBar["TopBar"]
            EngineTabs["EngineTabs"]
            Editor["SQLEditor"]
            Results["ResultsPanel"]
            Schema["SchemaExplorer"]
            ImportExport["DatabaseManagerModal / ExportModal"]
            LoadSimulator["LoadSimulatorModal"]
            History["HistoryModal"]
            Settings["SettingsModal / EnvModal"]
        end

        Store["useStore<br/>Estado global Zustand"]

        subgraph Engines["Motores simulados"]
            SQLEngine["sqlEngine.ts<br/>AlaSQL + preprocesadores"]
            MongoEngine["executeMongoQuery"]
            RedisEngine["executeRedisCommand"]
            ExportHelper["exportHelper.ts"]
        end

        subgraph Persistence["Persistencia local"]
            IDBStorage["idbStorage.ts"]
            QueryLogger["Historial y logs locales"]
            LocalSettings["localStorage"]
        end

        subgraph FirebaseServices["Servicios Firebase"]
            Auth["auth.ts"]
            Presence["presence.ts"]
            SimulatorSession["simulatorSession.ts"]
            Roles["Gestion de roles / usuarios"]
        end

        subgraph Automation["Automatizacion"]
            PerfScript["performance-test.mjs"]
            SummaryScript["performance-summary.mjs"]
            WorkflowPerf["performance.yml"]
            WorkflowPages["pages.yml"]
        end
    end

    IDB["IndexedDB"]
    BrowserStorage["LocalStorage"]
    Firebase["Firebase Auth / RTDB"]
    GitHub["GitHub Actions"]

    Usuario --> App

```

```
Usuario --> SimulatorView
Admin --> AdminApp

App --> UI
SimulatorView --> LoadSimulator
AdminApp --> FirebaseServices
UI --> Store
Store --> Engines
Store --> Persistence
Editor --> SQLEngine
ImportExport --> SQLEngine
Results --> ExportHelper
Schema --> IDBStorage
LoadSimulator --> SQLEngine
LoadSimulator --> SimulatorSession
History --> QueryLogger
Login --> Auth

IDBStorage --> IDB
QueryLogger --> BrowserStorage
LocalSettings --> BrowserStorage
Auth --> Firebase
Presence --> Firebase
SimulatorSession --> Firebase
Roles --> Firebase
WorkflowPerf --> PerfScript
WorkflowPerf --> SummaryScript
WorkflowPerf --> GitHub
WorkflowPages --> GitHub
```

B. Diagrama de Componentes - Sistema Desktop

El sistema desktop usa Electron como contenedor de la aplicación web, reutilizando la interfaz React y los servicios locales del navegador.

Diagrama Mermaid

flowchart TB

Usuario((Usuario Desktop))

subgraph Desktop["Sistema Desktop Electron"]

Main["electron/main.cjs
Proceso principal"]

Window["BrowserWindow
Ventana de escritorio"]

Menu["Menu / ciclo de vida"]

Assets["dist/
Build Vite"]

subgraph Renderer["Proceso renderer"]

ReactApp["React App"]

Components["Componentes UI"]

Store["Zustand Store"]

Engine["AlaSQL + simuladores
MongoDB / Redis"]

Exporter["ExportHelper / SheetJS"]

end

subgraph LocalData["Datos locales del escritorio"]

IDBStorage["IndexedDB Storage"]

LocalStorage["LocalStorage"]

Files["Archivos exportados
CSV / JSON / Excel / SQL"]

end

subgraph OptionalCloud["Servicios opcionales"]

Auth["Firebase Auth"]

RTDB["Firebase RTDB
presencia / sesiones / roles"]

end

end

OS["Sistema operativo"]

Firebase[("Firebase")]

Usuario --> Window

Main --> Window

Main --> Menu

Window --> Assets

Assets --> ReactApp

ReactApp --> Components

Components --> Store

Store --> Engine

Engine --> IDBStorage

Components --> Exporter

Exporter --> Files

Store --> LocalStorage

ReactApp --> Auth

ReactApp --> RTDB

Auth --> Firebase

RTDB --> Firebase

Files --> OS

IDBStorage --> OS

LocalStorage --> OS

X. VISTA DE DATOS

A. Diagrama Entidad Relacion

La vista de datos se representa mediante IndexedDB para tablas y esquemas locales, LocalStorage para preferencias y metadatos de sesion, y Firebase Realtime Database para usuarios, presencia y sesiones del simulador.

Diagrama Mermaid

erDiagram

```

USUARIO ||--o{ ENGINE_TAB : crea
ENGINE_TAB ||--o{ QUERY_PANE : contiene
ENGINE_TAB ||--o{ QUERY_RESULT : genera
ENGINE_TAB ||--o{ SCHEMA_ENTRY : registra
USUARIO ||--o{ QUERY_HISTORY : consulta
USUARIO ||--o{ SIMULATOR_SESSION : publica
SIMULATOR_SESSION ||--o{ LOAD_METRIC : produce
SIMULATION_SETTINGS ||--o{ LOAD_METRIC : configura

```

```

USUARIO {
    string id
    string email
    string role
    string displayName
}

```

```

ENGINE_TAB {
    string id
    string engine
    string database
    string activeQueryPaneId
}

```

```

QUERY_PANE {
    string id
    string query
    string engine
}

```

```

QUERY_RESULT {
    string id
    int rowCount
    int executionTime
    string status
}

```

```

SCHEMA_ENTRY {
    string dbName
    string tableName
    string columns
}

```

```

QUERY_HISTORY {
    string id
    string query
    string engine
    string executedAt
}

```

```

SIMULATION_SETTINGS {
    int networkLatency
    int connectionLimit
    boolean simulateErrors
    float errorProbability
}

```

```

SIMULATOR_SESSION {

```

```
    string id
    string userId
    string engine
    string status
}

LOAD_METRIC {
    string id
    int users
    float tps
    float latency
    float errorRate
}
```

XI. CALIDAD

A. Escenario de Seguridad

Escenario	Descripcion	Implementacion
ES001	Acceso administrativo controlado	Firebase Auth y roles para administracion.
ES002	Proteccion de credenciales	Variables de entorno fuera del repositorio publico.
ES003	Separacion de alcance	El simulador no se conecta a motores reales.

B. Escenario de Usabilidad

Escenario	Descripcion	Implementacion
EU001	Interfaz tipo IDE	Editor, tabs, sidebar, resultados y esquema.
EU002	Acciones visibles	Barra superior con importar, exportar, ayuda y simulador.
EU003	Aprendizaje guiado	Pantalla de bienvenida, plantillas y ayuda.
EU004	Visualizacion clara	Resultados tabulares, metricas y graficos de carga.

C. Escenario de Adaptabilidad

Escenario	Descripcion	Implementacion
EA001	Nuevos motores	Configuracion central de motores y exportadores.
EA002	Web y desktop	Vite para web y Electron para escritorio.
EA003	Servicios opcionales	Firebase se usa cuando existe configuracion disponible.

D. Escenario de Disponibilidad

Escenario	Descripcion	Implementacion
ED001	Operacion local	Consultas, importacion y exportacion funcionan sin backend propio.
ED002	Persistencia local	IndexedDB conserva tablas y esquemas.
ED003	Recuperacion de evidencias	Exportacion permite respaldar resultados y sesiones.

E. Escenario de Escalabilidad

Escenario	Descripcion	Implementacion
EE001	Carga en cliente	La ejecucion de consultas ocurre en el navegador.
EE002	Hosting estatico	El build puede publicarse en Netlify, Firebase Hosting o GitHub Pages.
EE003	Monitoreo externo	Firebase soporta presencia y sesiones de simulador para grupos academicos.

El sistema no busca medir rendimiento real de motores, sino ofrecer una experiencia fluida de laboratorio. El simulador de carga calcula TPS, CPU y latencia mediante formulas internas para representar escenarios didacticos.

CONCLUSIONES

1. La arquitectura del simulador separa correctamente interfaz, estado, motores, persistencia y servicios externos.
2. El uso de React, Vite y TypeScript permite una base mantenible y portable.
3. AlaSQL e IndexedDB hacen viable la ejecucion local sin backend propio.
4. Firebase agrega autenticacion, presencia y administracion cuando se requiere trabajo monitoreado.
5. Electron extiende el alcance del producto hacia escritorio.
6. La arquitectura es adecuada para fines academicos y puede crecer hacia conectores reales en una version futura.

RECOMENDACIONES

1. Agregar pruebas unitarias para `sqlEngine`, importadores y exportadores.
2. Corregir textos con problemas de codificacion para una presentacion profesional.
3. Documentar configuracion Firebase en un archivo de guia.
4. Agregar capturas o anexos visuales de cada vista principal.
5. Mantener separada cualquier futura conexion real para no confundir el alcance del simulador.
6. Definir limites recomendados de tamanos de dataset para evitar sobrecargar el navegador.

Documento preparado por: Jhony Vargas Luque y Abel Fernando Pacompia Ortiz

Fecha de elaboracion: 21 de junio de 2026

Version: 2.0

Estado: Aprobado